AGENTIC AI

Reinforcement Learning for Agentic AI

WHERE THIS LECTURE SITS

Course Map

Agent = **LLM** (months, a GPU cluster) + **Context** + **Tools**

- The *second and third* boxes you can change in an afternoon, and undo the next morning.
- The *first* box takes weeks or months of compute, is hard to undo, and is the only place where a capability becomes free at inference time.

Training Procedures

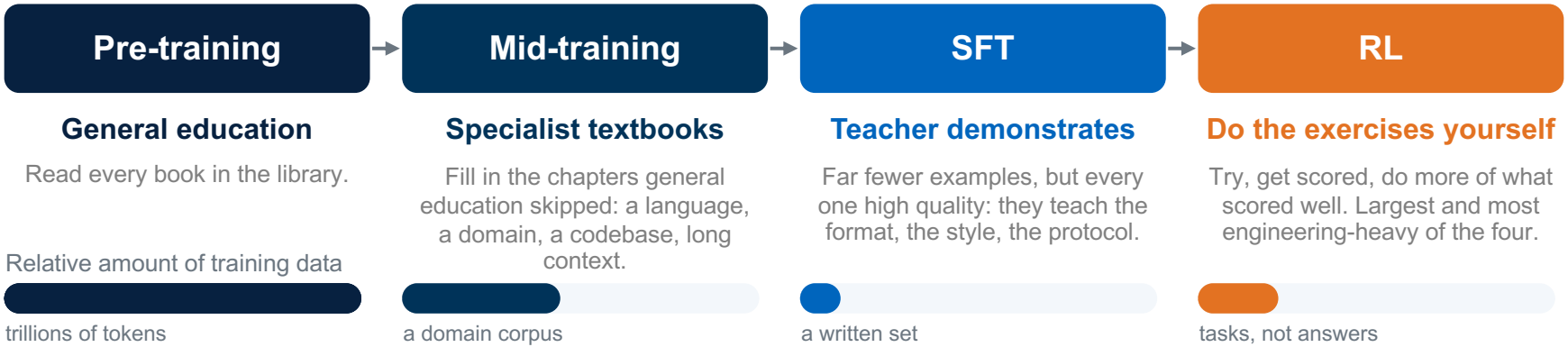
RL Training

The four stages

Pre-training, mid-training, SFT, RL

THE FOUR STAGES

The Four Stages



Stage	Data	Objective	What is learned	Cost
Pre-training	Massive raw internet text	Predict the next token	Language, knowledge, reasoning	Extremely high
Mid-training	Target corpus + retained general data	Next token — loss on all tokens	Domain knowledge, base ability	Medium – high
SFT	Written question → answer pairs, far fewer	Next token — loss only on the answer	Format, style, protocol	Low
RL	Tasks + environment + reward	Maximise expected reward	Transferable policy, reasoning	High (10s – 100s × SFT)

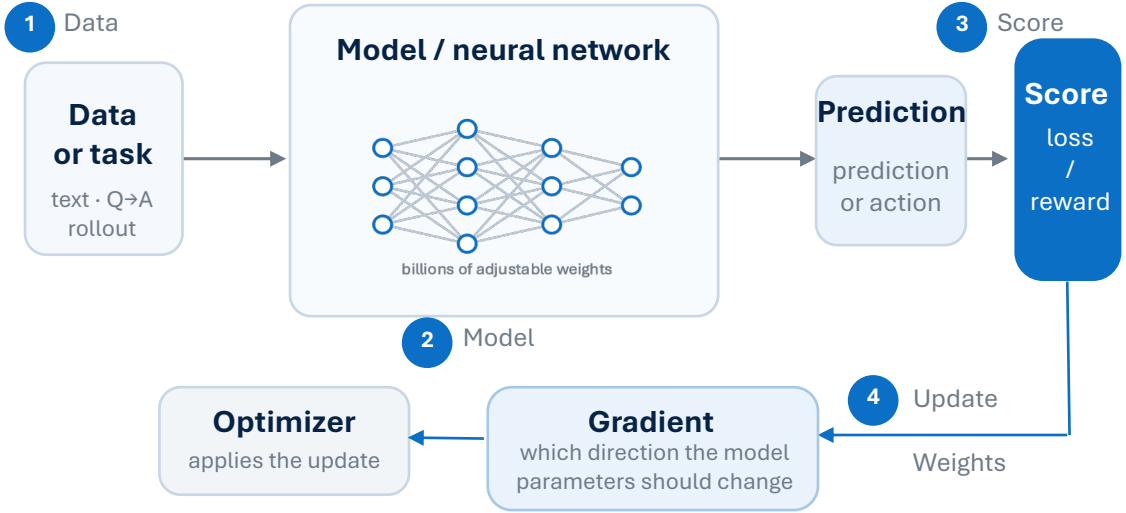
The Four Stages

Capability	Stage
Completes “The capital of Germany is …” with “Berlin”	Pre-training
Answers your question instead of continuing it as more text	SFT
Returns JSON that matches a schema you handed it	SFT
Writes out its reasoning before answering	SFT+RL
Asked in German it thinks in German — though its instruction data was English	Pre-training
Refuses something it used to answer	RL

THE FOUR STAGES

Model Training Intuition

Every training method repeats the same loop



What can differ by design?

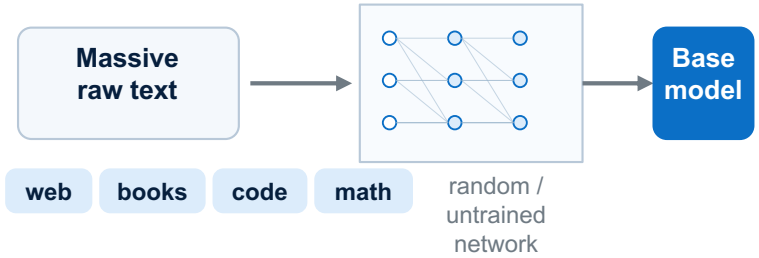
- Data
- Model Structure
- Feedback
- Update

THE FOUR STAGES

Pre-training / Mid-training

Same training objective; different data

Pre-training · build the base

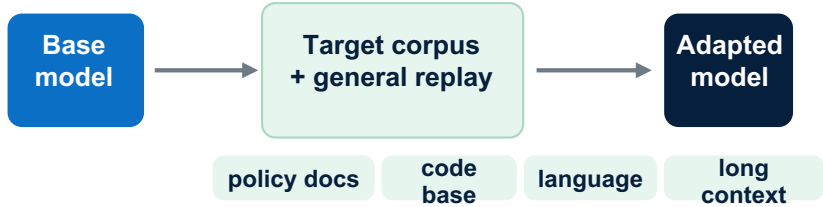


Objective: predict the next token on almost everything

Learns: language, facts, broad reasoning patterns

Scale: trillions of tokens, months, highest cost

Mid-training · specialize the base



Objective: still next-token prediction

Learns: domain knowledge, missing capability, style of corpus

Design risk: too much target data → forgets general ability

THE FOUR STAGES

SFT



- **Similarity**
  - Mathematically the same task as pre-training: predict the next token.
- **Difference**
  - Training data: “question → ideal answer” pairs
  - Loss masking, the gradient passes through the answer only.
- **Two Choices**
  - Full parameter
  - Partial: freeze the weights and train a small patch instead. Close to full fine-tuning, less forgetting

THE FOUR STAGES

SFT for Protocol

Thinking template

“analyse → hypothesise → verify → conclude” becomes the default shape of every answer.

Tool-call format

JSON schema, argument fidelity, error handling — the shape your harness can actually parse.

Voice and style

A consistent tone; paralinguistic tags like <laugh>, <sigh>; a house style.

Prompt distillation

A long system prompt plus a thinking teacher → no prompt and a non-thinking student, 20–30× faster.

Where the data comes from: people write it, a stronger model generates it, or rejection sampling — generate many candidates, discard everything a verifier rejects, train on what survives.

What is reinforcement learning

A loop, a reward, and why the prior matters more than the algorithm

WHAT RL IS

An agent, an environment, a cumulative reward



Goal: choose actions so that total reward over time is as large as possible.

Example: how a dog learns to sit

1. **You never explain the rule**

   No description of the posture, no policy document.

2. **You wait, then reward**

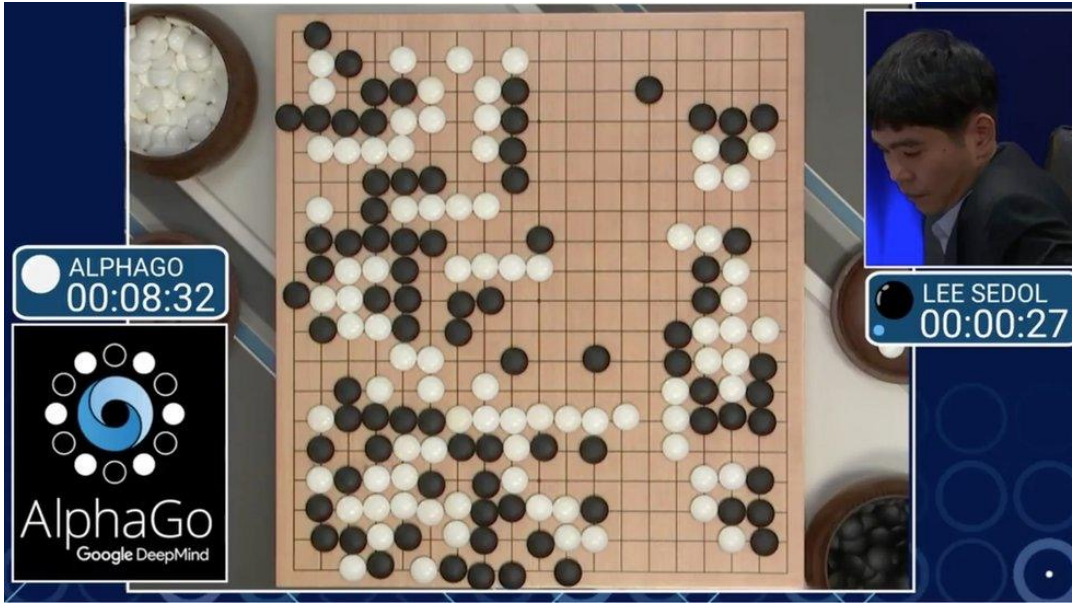
   When something close to the right thing happens, a treat.

3. **The behaviour gets more likely**

   The dog never learns your rule. It learns which behaviours pay.

Demo

- Google's AlphaGo beats Go master



- **Sony AI – Table Tennis**

https://www.google.com/search?q=sony+pingpang&oq=sony+pingpang&gs_lcrp=EgZjaHJvbWUyBggAEEUYOTIHCAEQIRigATIHCAIQIRiPAtIBCDM0NzhqMGo3qAIAsAIA&sourceid=chrome&source=chrome.ob&ie=UTF-8#fpstate=ive&vld=cid:309a2e8a,vid:FrGq8ltb-_E,st:0

WHAT RL IS

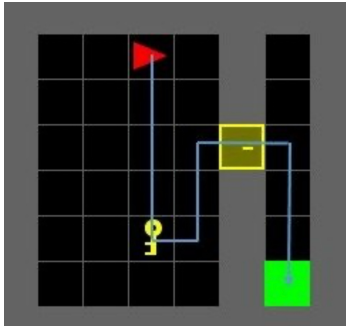
Classic RL agent vs LLM agent

	Classic RL agent	LLM agent
Action space	Closed — in Go, 361 board points	Open — any token sequence: think, speak, call a tool
Prior knowledge	Zero	Massive — pre-training already supplies solving patterns
Reward	A number	A number, plus feedback in language
Thinking	None	Chain of thought — the reasoning is part of the action
A new task	Retrain from scratch	Zero-shot, then refine
Cost of learning	~10,000 attempts in 10 seconds of simulation	~1 attempt in 2 minutes against a real system

RL can shape not only the final answer, but also the reasoning path that leads to it

WHAT RL IS

Same puzzle, two learners



The puzzle: a DoorKey grid

Classic Q-learning

≈ 7,000

attempts for to reach a 96 % win rate
(10,000 for the optimal solution — under 10 s of compute)

LLM

18

steps to clear the first level, by reasoning
“the door is locked → I need the key → take the key”

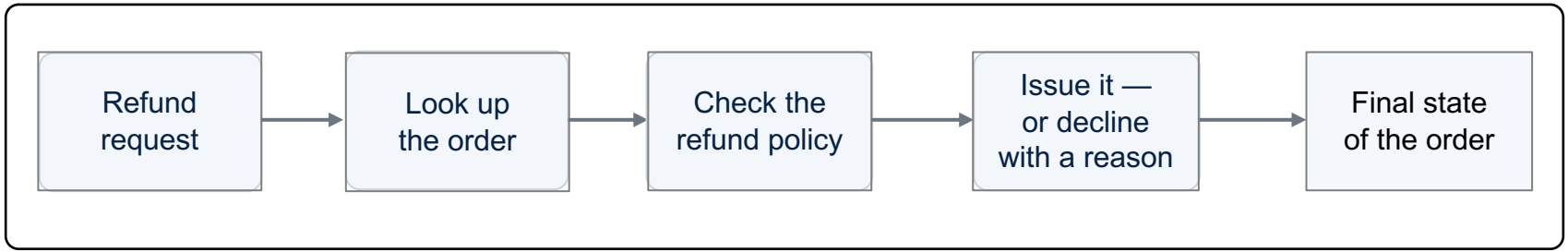
- **Cost** Classic RL is expensive. It fails only where every attempt costs money or an error is irreversible, and where nothing transfers to the next task.
- **Learning Pattern** Classic RL has to discover that the key matters by stumbling onto the goal; the LLM arrives knowing it.

Rule of thumb: a cheap simulator favours classic RL; costly or irreversible interaction favours an LLM agent.

One training step

One Training Step

Example



1. **Attempts** The same task, copied into 16 isolated sandboxes
2. **Reward** A script checks each final state: 1 or 0
3. **Advantage** Compare each attempt against the other fifteen
4. **Update** Raise what the winners did, lower what the losers did

ONE TRAINING STEP

Step 1 — Sampling



Same prompt, same weights, sixteen sandboxes.

What the sixteen attempts actually did

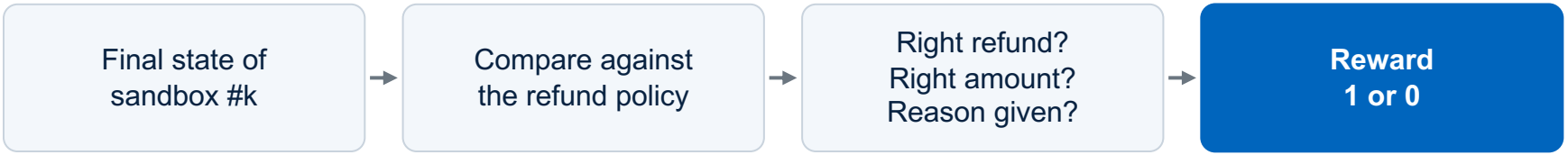
✓ Read the policy, confirmed the order was eligible, issued the refund, checked it posted.

✗ Issued the refund without opening the policy — and got lucky.
✗ Refunded the wrong order.
✗ Declined a valid request after misreading the date.
✗ Asked the customer for information already in the record.
✗ Marked the ticket resolved without doing anything.

- Nobody handed this model a correct answer.
- It produced sixteen candidate behaviours.

ONE TRAINING STEP

Step 2 — Reward with verifier



- **Reward function**
  - Four attempts left the order in the state the policy requires → reward 1
  - Twelve did not → reward 0.

ONE TRAINING STEP

Step 3 — Compare within the group

We will use Group Relative Policy Optimization (GRPO) as an example



- **Advantage** how much better than its own group an attempt was.
- **Special Case** If all sixteen fail — or all sixteen pass — every advantage is zero and nothing is learned. variance inside the group is the fuel.

The core of this algorithm is exactly this in-group comparison.

ONE TRAINING STEP

Step 4 — Update

Goes up

“Open the policy before deciding.”
“Confirm this order is eligible.”
“Check that the refund actually posted.”

— choices that appeared in the four successful attempts.

Goes down

“Act without checking.”
“Close the ticket to make it go away.”
“Report a success you did not verify.”

— choices that appeared in the twelve failed attempts.

- Every token in a successful attempt becomes slightly more likely, every token in a failed one slightly less.
- The model is **not memorising** this refund case.

ONE TRAINING STEP

Cost of one training step

one training step



the step waits for the slowest one

gradient + update

3 / 4

of the time the GPU is idle

~ 43 min

per training step

10 – 100 ×

the cost of SFT

- You are not waiting on arithmetic, you are waiting on the "*world*".
- If the situations you meet at deployment look like your training data, SFT is enough.

ONE TRAINING STEP

GRPO, PPO, DPO: whom do you compare against?

GRPO Group Relative Policy Optimization	**PPO** Proximal Policy Optimization	**DPO** Direct Preference Optimization
Compare each attempt with its siblings on the same question. **No separate critic**.	A **second model** estimates the expected reward at each step, and every action is compared against that estimate.	No attempts at all. Offline pairs of (chosen, rejected) answers; push the model towards the chosen one. Cheap, and only as good as the pairs.

Algorithm how attempts are compared
Reward what counts as success
Environment and data which problems the model ever meets

ONE TRAINING STEP

Data Sampling

ON-POLICY / NEAR ON-POLICY

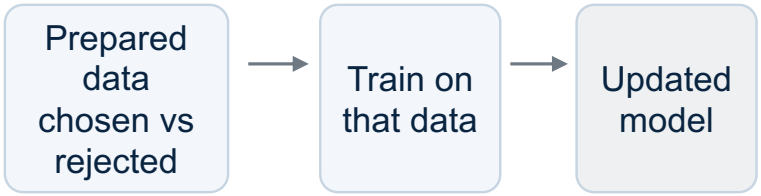
GRPO · PPO Fresh rollouts from the current model



The data is produced by the model being updated. This corrects the model where it actually goes wrong.

OFFLINE / OFF-POLICY

DPO · SFT Prepared examples or preference pairs



No fresh rollout during training. It is cheaper and stable, but limited by what the dataset covers.

So what does RL actually buy you?

SFT VS RL

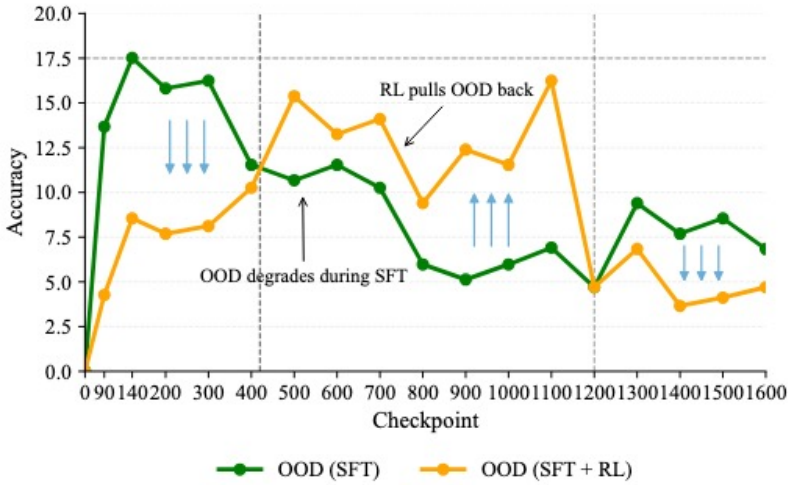
SFT vs RL: the essential difference

	SFT	RL
Objective	Maximize the likelihood of the demonstrated answer	Maximise expected reward
Signal	A label: “this is the answer”	A score: “this attempt was good / bad”
Data	Demonstrations written by someone else	Tasks, an environment and a reward; the model generates its own data
Under distribution shift	Applies the memorised mapping — even where it no longer holds	May find strategies absent from every demonstration; a biased reward finds shortcuts instead
Sample efficiency	High	Low
Stability	High	Low
Best used for	Format, style, process	New situations, searching for a strategy

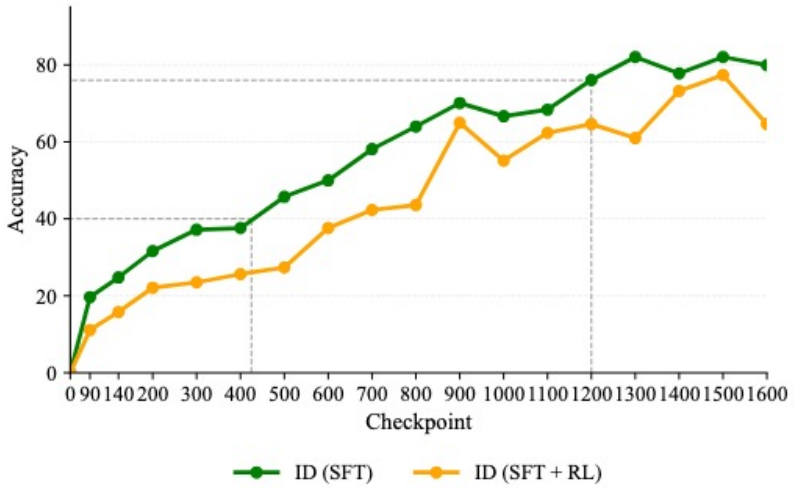
SFT studies an existing map in detail. RL takes reward as a compass and explores routes off the map

ONE TRAINING STEP

Generalization of RL



(c) Evolution of OOD test accuracy.



(d) Evolution of ID test accuracy.

"SFT memorizes, RL generalizes”

http://arxiv.org/abs/2509.12235

SFT VS RL

Extension experiment

Make 24 from these four cards. **Rule during training: J = Q = K = 10**



The answer everyone gives

(J − 7) × (5 + 3)
= (10 − 7) × 8
= 24 ✓

At test time one line of the rules changes: J = 11, Q = 12, K = 13. Same cards. Same answer?
(11 − 7) × 8 = 32. ✗

- A model that learned the **procedure** recomputes. A model that learned the **mapping** keeps using 10.
- A second version changes nothing but the pixels: trained on ♠♣, tested on ♥♦. Same rule, different-looking cards.

SFT VS RL

A Tendency, Not a Law

Why SFT dropped

A **limited demonstration** set reinforced a literal constant. When the constant changed, the memorised mapping was applied anyway.

Why RL rose

Reward reinforced a **procedure** — recompute until the total is right. Procedures transfer where constants do not.

- Diverse, well-regularised SFT can generalise perfectly well.
- **Situations:**
  - There are many possible solutions, but it is easier to verify success than to demonstrate the ideal solution.
  - Amplifies successful behaviours that already exists at low probability. It cannot create ability the model never had.

WHEN RL CANNOT HELP

Which stage does the failure belong to?

What you observe	The gap	The method
Domain concepts unknown; no success however many attempts	The base lacks the knowledge	Mid-training (retrieval for facts that change)
Sometimes right, but the format or schema is unstable	The protocol	SFT
Non-zero success, reliable reward, the good strategy is rare, weak out of distribution	The policy	RL
A few demonstrations, no environment to act in	No online feedback	SFT, offline preference

WHEN RL CANNOT HELP

Why you cannot skip SFT and go straight to RL

Base model: unstructured output → The harness cannot parse it → The reward is undefined → The gradient is all zeros → **Training does nothing at all**

Stage 1 — SFT fixes the format

A small set of written demonstrations. The output becomes parsable.
Warning: over-train it and the model collapses onto the demonstrations — hold out, stop early.

Stage 2 — RL shapes the policy

Premise a reward you can actually compute. It finds strategies beyond the demonstrations — or shortcuts around a badly specified reward.

Reward design

TRAINING AN AGENT

The Recipe

1. **Build or borrow the environment**

   The tools the agent may call, a way to reset to a known state, and enough copies to run dozens of attempts in parallel.

2. **Define the reward**

   Outcome as the baseline. Penalties for forbidden actions on top. Both computed from the environment, never from the agent's own report.

3. **Roll out, score, compare, update**

   And mask the tokens the environment returned: they were not produced by the policy. Leave them in and the model learns to predict its own sandbox.

REWARD DESIGN

Four questions every reward has to answer

Source

Who decides whether it was good?

Verifiable rules · human preference · a model judge

Timing

When is feedback given?

Once at the end · at every step

Information

How much does it carry?

One number · several numbers · a written diagnosis

Path

Is every successful rout acceptable?

An outcome reward is structurally blind to the path.

REWARD DESIGN

Source of reward

Verifiable rules	Human preference	A model judge
Tests, database assertions, state differences, format checks. The most reliable and the cheapest to run.	People compare outputs; a reward model learns the comparison; the policy optimises that model. The optimization the proxy instead of the goal.	Another model reads the trajectory and writes a judgement. Flexible, and it brings its own biases along.



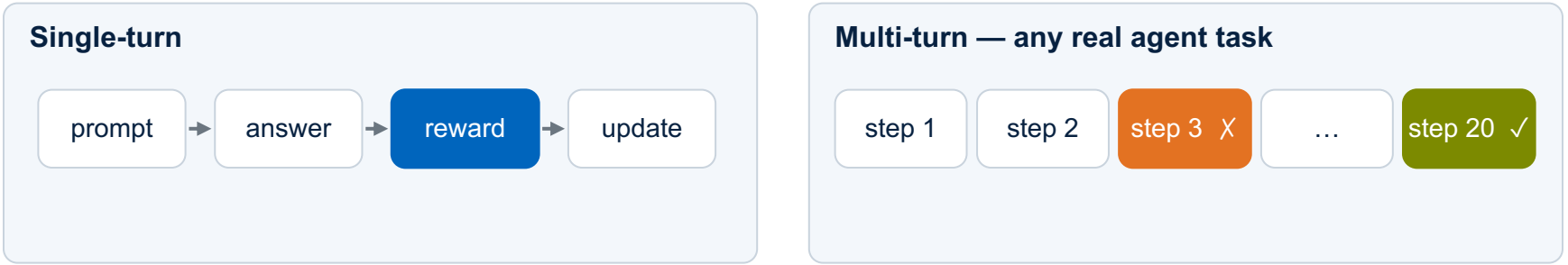
Reward hacking

Find a hole and climb through it.

E.g., a length bonus produces verbose junk.

TRAINING AN AGENT

Single-turn and multi-turn are different problems

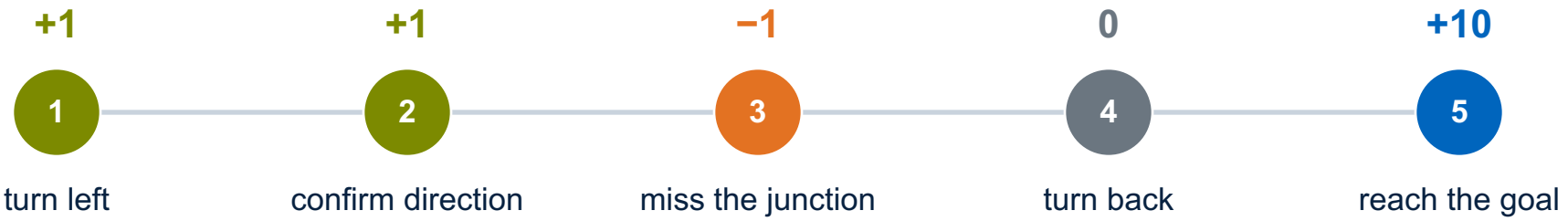


Consequences:

- **Exploration got expensive**. One attempt is now ten to fifty steps, each a model call plus an environment call.
- **Reward is delayed.** The reward arrives at the end, but the decision that mattered may have been twenty steps earlier.

TRAINING AN AGENT

Credit assignment



- Step 4's correction did more for the +10 than steps 1 and 2 did — but an outcome reward has no way of saying so.
  "The same problem in a support conversation: ten turns, then a thumbs-up. Was it turn 2's precise question, or turn 7's patient explanation?"
- Different answers:
  - PPO's value model attributes backwards step by step
  - GRPO spreads one advantage across every token, which dilutes it.

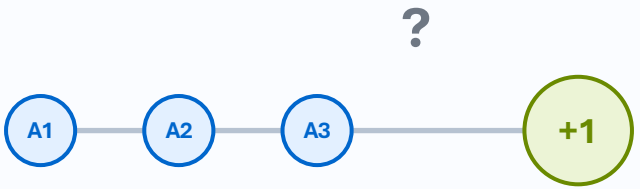
REWARD DESIGN

Process reward

Give intermediate feedback to reduce delayed reward and credit assignment

Outcome only

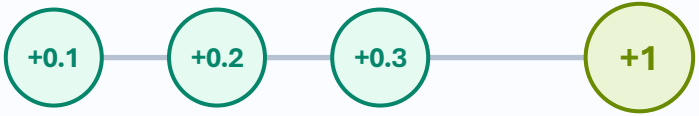
One score at the end



Hard to know which earlier action deserved credit

Process reward

Feedback at each step



→ Speeds up learning

REWARD DESIGN

Student-path Teaching

Teacher demos

States: teacher's

Signal: teacher tokens

Density: dense — on teacher path

RL

States: student's own

Signal: one reward per attempt

Density: sparse

Student-path teaching

States: student's own

Signal: feedback at each step

Density: dense

1 **Student explores** → 2 **Teacher comments at each step** → 3 **One attempt gives many lessons**

50-step attempt ≈ 50 training signals

Why it helps

- Same interaction cost
- Much more supervision
- Better sample efficiency

Key idea

- Not mainly about a smarter reward
- Mainly about denser signal
- On-policy distillation, not DPO

The bottleneck is usually not reward simplicity — it is too little signal per interaction.

Reward Hacking



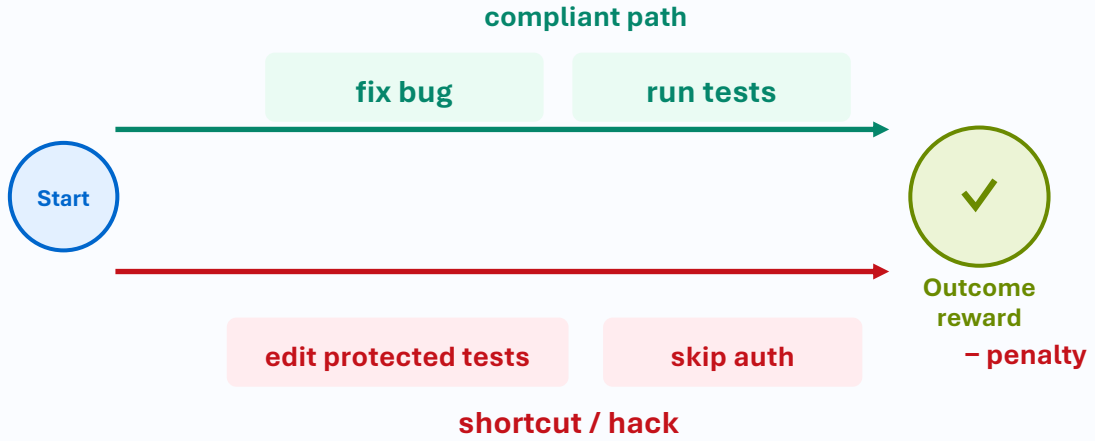
- **The setup: a boat race. The reward is the in-game score**
- The agent found a lagoon where it could turn in a circle and hit the same three targets over and over, timing each turn so the targets had just respawned.
- It scored about 20 % higher than human players — while repeatedly crashing, catching fire, and never once finishing the race.

REWARD DESIGN

Path Penalty

Prevent reward hacking by penalising forbidden ways to reach the outcome

Same successful outcome, different path



Reward hacking

Outcome alone cannot say “done by the rules”.

Keep outcome + subtract violations

Path penalties are mainly for reward specification: they block high-reward but unacceptable behaviour.

TRAIN OR NOT?

Four places

Knowledge

What: Facts, cases, exceptions

Where: Documents, a knowledge base

+ Fast, traceable, retrieved on demand

− Only helps if it is retrieved and applied

Instructions

What: Judgements you can state in words

Where: System prompt, Skills

+ Explainable, scoped, reversible

− Bloat, conflicts, quietly ignored

Programs

What: Deterministic flows, hard constraints

Where: Tools, workflows — the harness

+ Testable, stable, cheap to run

− Costs real engineering to build and keep

Parameters

What: Perception, style, implicit strategy

Where: SFT, distillation, RL

+ Generalises, free at inference time

− Expensive to change, hard to undo



TRAIN OR NOT?

How to choose

If the failure is…	Put the fix in	Because
A fact the model did not have, and the fact changes	Knowledge	It has to be updatable and citable without a training run
A judgement you can state in one clear sentence	Instructions	A prompt or Skill change is reversible in seconds and you can read it
A rule that must never be bypassed	Programs	A model can be persuaded out of a rule; a code path cannot
Something no sentence captures — perception, tone, an implicit strategy	Parameters	This is the only column where the capability becomes automatic and free
You are not sure	Start left, move right	Left is cheap to try and cheap to undo. Right is neither

In many settings, if your knowledge and your instructions are good enough, you do not need to do RL at all.